13
Geometry

Above the gateway to Plato's academy appeared the inscription, "Let no one who is ignorant of geometry enter here." The organizers of programming competitions feel much the same way, for you can count on seeing at least one geometric problem at every contest.

Geometry is an inherently visual discipline, one that mandates drawing pictures and studying them carefully. Part of the difficulty of geometric programming is that certain "obvious" operations you do with a pencil, such as finding the intersection of two lines, requires non-trivial programming to do correctly with a computer.

Geometry is a subject which everybody studies in high school but which often turns rusty with time. In this chapter, we will refresh your knowledge with programming problems associated with "real" geometry – lines, points, circles, and so forth. After solving a few of these problems you should feel confident enough to walk through Plato's academy once again.

There is more geometry to come. We defer problems associated with line segments and polygons to Chapter 14.

13.1 Lines

Straight *lines* are the shortest distance between any two points. Lines are of infinite length in both directions, as opposed to *line segments*, which are finite. We limit our discussion here to lines in the plane.

- *Representation* — Lines can be represented in two different ways, as either pairs of points or as equations. Every line l is completely represented by any pair of

points (x_1, y_1) and (x_2, y_2) which lie on it. Lines are also completely described by equations such as $y = mx + b$, where m is the *slope* of the line and b is the *y-intercept*, i.e., the unique point $(0, b)$ where it crosses the x-axis. The line l has slope $m = \Delta y/\Delta x = (y_1 - y_2)/(x_1 - x_2)$ and intercept $b = y_1 - mx_1$.

Vertical lines cannot be described by such equations, however, because dividing by Δx means dividing by zero. The equation $x = c$ denotes a vertical line that crosses the x-axis at the point $(c, 0)$. This special case, or *degeneracy*, requires extra attention when doing geometric programming. We use the more general formula $ax + by + c = 0$ as the foundation of our line type because it covers all possible lines in the plane:

```
typedef struct {
        double a;               /* x-coefficient */
        double b;               /* y-coefficient */
        double c;               /* constant term */
} line;
```

Multiplying these coefficients by any non-zero constant yields an alternate representation for any line. We establish a canonical representation by insisting that the y-coefficient equal 1 if it is non-zero. Otherwise, we set the x-coefficient to 1:

```
points_to_line(point p1, point p2, line *l)
{
        if (p1[X] == p2[X]) {
                l->a = 1;
                l->b = 0;
                l->c = -p1[X];
        } else {
                l->b = 1;
                l->a = -(p1[Y]-p2[Y])/(p1[X]-p2[X]);
                l->c = -(l->a * p1[X]) - (l->b * p1[Y]);
        }
}

point_and_slope_to_line(point p, double m, line *l)
{
        l->a = -m;
        l->b = 1;
        l->c = -((l->a*p[X]) + (l->b*p[Y]));
}
```

- *Intersection* — Two distinct lines have one *intersection point* unless they are *parallel*; in which case they have none. Parallel lines share the same slope but have different intercepts and by definition never cross.

```
bool parallelQ(line l1, line l2)
{
     return ( (fabs(l1.a-l2.a) <= EPSILON) &&
              (fabs(l1.b-l2.b) <= EPSILON) );
}

bool same_lineQ(line l1, line l2)
{
     return ( parallelQ(l1,l2) && (fabs(l1.c-l2.c) <= EPSILON) );
}
```

A point (x', y') lies on a line $l : y = mx + b$ if plugging x' into the formula for x yields y'. The intersection point of lines $l_1 : y = m_1 x + b_1$ and $l_2 : y_2 = m_2 x + b_2$ is the point where they are equal, namely,

$$x = \frac{b_2 - b_1}{m_1 - m_2}, \qquad y = m_1 \frac{b_2 - b_1}{m_1 - m_2} + b_1$$

```
intersection_point(line l1, line l2, point p)
{
     if (same_lineQ(l1,l2)) {
         printf("Warning: Identical lines, all points intersect.\n");
         p[X] = p[Y] = 0.0;
         return;
     }

     if (parallelQ(l1,l2) == TRUE) {
         printf("Error: Distinct parallel lines do not intersect.\n");
         return;
     }

     p[X] = (l2.b*l1.c - l1.b*l2.c) / (l2.a*l1.b - l1.a*l2.b);

     if (fabs(l1.b) > EPSILON)       /* test for vertical line */
             p[Y] = - (l1.a * (p[X]) + l1.c) / l1.b;
     else
             p[Y] = - (l2.a * (p[X]) + l2.c) / l2.b;
}
```

- *Angles* — Any two non-parallel lines intersect each other at a given angle. Lines $l_1 : a_1 x + b_1 y + c_1 = 0$ and $l_2 : a_2 x + b_2 y + c_2 = 0$, written in the general form, intersect at the angle θ given by:

$$\tan\theta = \frac{a_1 b_2 - a_2 b_1}{a_1 a_2 + b_1 b_2}$$

For lines in slope-intercept form this reduces to $\tan\theta = (m_2 - m_1)/(m_1 m_2 + 1)$.

Two lines are *perpendicular* if they cross at right angles to each other. For example, the x-axis and y-axis of a rectilinear coordinate system are perpendicular, as are the lines $y = x$ and $y = -1/x$. The line perpendicular to $l : y = mx + b$ is $y = (-1/m)x + b'$, for all values of b'.

- *Closest Point* — A very useful subproblem is identifying the point on line l which is closest to a given point p. This closest point lies on the line through p which is perpendicular to l, and hence can be found using the routines we have already developed:

```
closest_point(point p_in, line l, point p_c)
{
      line perp;              /* perpendicular to l through (x,y) */

      if (fabs(l.b) <= EPSILON) {     /* vertical line */
              p_c[X] = -(l.c);
              p_c[Y] = p_in[Y];
              return;
      }

      if (fabs(l.a) <= EPSILON) {     /* horizontal line */
              p_c[X] = p_in[X];
              p_c[Y] = -(l.c);
              return;
      }

      point_and_slope_to_line(p_in,1/l.a,&perp); /* normal case */
      intersection_point(l,perp,p_c);
}
```

- *Rays* — These are half-lines originating from some vertex v, called the *origin*. Any ray is completely described by a line equation, origin, and direction or the origin and another point on the ray.

13.2 Triangles and Trigonometry

An *angle* is the union of two rays sharing a common endpoint. *Trigonometry* is the branch of mathematics dealing with angles and their measurement.

There are two common units used to measure angles, *radians* and *degrees*. The entire range of angles spans from 0 to 2π radians or, equivalently, 0 to 360 degrees. Using radians is better computationally, because the trigonometric libraries we will see in Section 13.5 assume angles are measured in radians. However, we confess that we think more naturally in degrees. Historically, fractional parts of angles measured in degrees are given in *minutes*, or 1/60th of a degree. But working in degrees and minutes is hopeless, which is why radians (or at least decimal degrees) are the preferred measure.

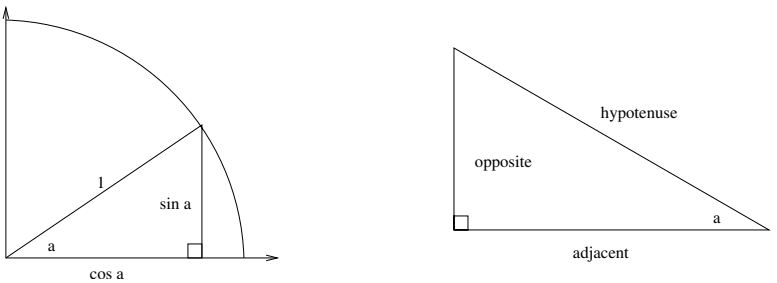


Figure 13.1. Defining sine and cosine (l). Labeling the edges of a right triangle (r).

The geometry of triangles ("three angles") is intimately related to trigonometry, so we will discuss them together in the sections below.

13.2.1 Right Triangles and the Pythagorean Theorem

A *right* angle measures 90° or $\pi/2$ radians. Right angles are formed at the intersection of two perpendicular lines, such as rectilinear coordinate axes. Such lines divide the $360^o = 2\pi$ radian space into four right angles.

Each pair of rays with a common endpoint actually defines two angles, an *internal angle* of a radians and an *external angle* of $2\pi - a$ radians. The internal angles are the ones we usually claim to be interested in. The three internal (smaller) angles of any triangle add up to $180^o = \pi$ radians, meaning that the average internal angle is $60^o = \pi/3$ radians. Triangles with three equal angles are called *equilateral*, as was discussed in Section 12.2.

A triangle is called a *right* triangle if it contains a right internal angle. Right triangles are particularly easy to work with because of the *Pythagorean theorem*, which enables us to calculate the length of the third side of any triangle given the length of the other two. Specifically, $|a|^2 + |b|^2 = |c|^2$, where a and b are the two shorter sides, and c is the longest side or *hypotenuse*.

One can go a long way in analyzing triangles with just the Pythagorean theorem. But we can go even farther using trigonometry.

13.2.2 Trigonometric Functions

The trigonometric functions *sine* and *cosine* are defined as the x- and y-coordinates of points on the unit circle centered at $(0, 0)$, as shown in Figure 13.1(l). Thus the values of sine and cosine range from -1 to 1. Further, the two functions are really the same thing, since $\cos(\theta) = \sin(\theta + \pi/2)$.

A third important trigonometric function is the *tangent*, defined as the ratio of sine over cosine. Thus $\tan(\theta) = \sin(\theta)/\cos(\theta)$, which is well-defined except when $\cos(\theta) = 0$ at $\theta = \pi/2$ and $\theta = 3\pi/2$.

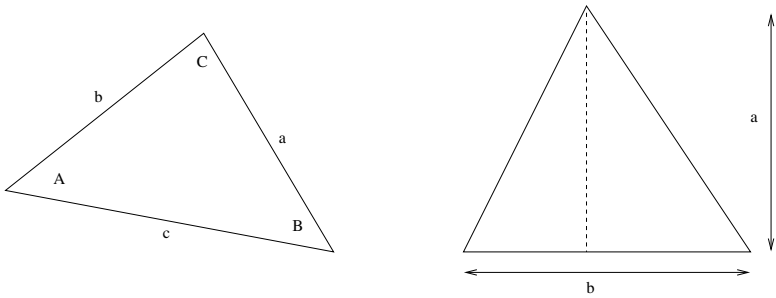


Figure 13.2. Notation for solving triangles (l) and computing their area (r).

These functions are important, because they enable us to relate the lengths of any two sides of a right triangle T with the non-right angles of T. Recall that the hypotenuse of a right triangle is the longest edge in T, that edge opposite the right angle. The other two edges in T can be labeled as *opposite* or *adjacent* edges in relation to a given angle a, as shown in Figure 13.1(r). Then

$$\cos(a) = \frac{|\text{adjacent}|}{|\text{hypotenuse}|}, \quad \sin(a) = \frac{|\text{opposite}|}{|\text{hypotenuse}|}, \quad \tan(a) = \frac{|\text{opposite}|}{|\text{adjacent}|}$$

These relations are worth remembering. As a mnemonic, we use the name of the famous Indian Chief Soh-Cah-Toa, where each syllable of his name encodes a different relation. "Cah" means *C*osine equals *A*djacent over *H*ypotenuse, for example.

Chief Soh-Cah-Toa would not be of much use without inverse functions mapping $\cos(\theta)$, $\sin(\theta)$, and $\tan(\theta)$ back to the original angles. These inverse functions are called arccos, arcsin, and arctan, respectively. With them, we can easily compute the remaining angles of any right triangle given two side lengths.

These trigonometric functions are properly computed using Taylor series expansions, but don't worry: the math library of your favorite programming language already includes them. Trigonometric functions tend to be numerically unstable, so be careful. Do not expect that θ exactly equals $\arcsin(\sin(\theta))$, particularly for large or small angles.

13.2.3 Solving Triangles

Two powerful trigonometric formulae enable us to compute important properties of triangles. The *Law of Sines* provides the relationship between sides and angles in any triangle. For angles A, B, C, and opposing edges a, b, c (shown in Figure 13.2(l)),

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C}$$

The *Law of Cosines* is a generalization of the Pythagorean theorem beyond right angles. For any triangle with angles A, B, C, and opposing edges a, b, c,

$$a^2 = b^2 + c^2 - 2bc\cos A$$

Solving triangles is the art of deriving the unknown angles and edge lengths of a triangle given a subset of such measurements. Such problems fall into two categories:

- *Given two angles and side, find the rest* — Finding the third angle is easy, since the three angles must sum to $180^o = \pi$ radians. The Law of Sines then gives us a way to find the missing edge lengths.

- *Given two sides and an angle, find the rest* — If the angle lies between the two edges, the Law of Cosines gives us the way to find the remaining edge length. The Law of Sines then enables us to mop up the unknown angles. Otherwise, we can use the Law of Sines and the angle sum property to determine all angles, and then the Law of Sines to get the remaining edge length.

The area $A(T)$ of a triangle T is given by $A(T) = (1/2)ab$, where a is the altitude and b is the base of the triangle. The base is any one of the sides while the altitude is the distance from the third vertex to this base, as shown in Figure 13.2(r). This altitude can be easily calculated via trigonometry or the Pythagorean theorem, depending on what is known about the triangle.

Another approach to computing the area of a triangle is directly from its coordinate representation. Using linear algebra and determinants, it can be shown that the *signed* area $A(T)$ of triangle $T = (a, b, c)$ is

$$2 \cdot A(T) = \begin{vmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ c_x & c_y & 1 \end{vmatrix} = a_x b_y - a_y b_x + a_y c_x - a_x c_y + b_x c_y - c_x b_y$$

This formula generalizes nicely to compute $d!$ times the volume of a simplex in d dimensions.

Note that the signed areas can be negative, so we must take the absolute value to compute the actual area. This is a feature, not a bug. We will see how to use the sign of this area to build important primitives for computational geometry in Section 14.1.

```
double signed_triangle_area(point a, point b, point c)
{
        return( (a[X]*b[Y] - a[Y]*b[X] + a[Y]*c[X]
                - a[X]*c[Y] + b[X]*c[Y] - c[X]*b[Y]) / 2.0 );
}

double triangle_area(point a, point b, point c)
{
        return( fabs(signed_triangle_area(a,b,c)) );
}
```